

# Capitolo 22 — PROC-006 — Memory Consolidation & Consistency Loop

|                    |                                                                                                         |
|--------------------|---------------------------------------------------------------------------------------------------------|
| <b>Pubblico</b>    | Lettori tecnici e non tecnici del WCM Process & Memory Book                                             |
| <b>Versione</b>    | FROZEN-22                                                                                               |
| <b>Data master</b> | 2026-08-29                                                                                              |
| <b>Stato</b>       | FROZEN                                                                                                  |
| <b>Source path</b> | wcm/documentation/process-memory-book/<br>chapters/22_proc_006_memory_consolidation_consistency_loop.md |
| <b>Source SHA</b>  | 862789e                                                                                                 |
| <b>Release</b>     | 2026-08-29T23:32:17.195Z                                                                                |

*GitHub main è la source of truth. Questo PDF è una release derivata: non costituisce approvazione né autorità.*

**Parte VI — Il Libro dei Processi WCM Stato:** FROZEN **Data:** 2026-08-29 **Scope:** WCM generale, domain-agnostic

## 22.0 Ricordare non significa conservare tutto

Un sistema che lavora nel tempo accumula continuamente differenze tra ciò che sapeva prima e ciò che sa adesso. Una decisione viene presa. Uno stato cambia. Un output viene approvato. Un workflow completa una transizione. Una nuova evidenza modifica ciò che è noto.

Il problema non è salvare tutto. È capire **che cosa deve sopravvivere**, dove deve essere registrato e quali altre rappresentazioni devono restare coerenti con quel cambiamento.

**PROC-006 — Memory Consolidation & Consistency Loop** governa questo passaggio.

La sua domanda fondamentale è:

**quale delta materiale deve passare dalla memoria viva alla memoria organizzativa, e come verifichiamo che il sistema non resti diviso tra versioni incompatibili dello stesso fatto?**

Il principio è:

PERSISTERE UN DELTA  
≠  
CONSOLIDARE LA MEMORIA

La scrittura è soltanto una parte del processo. Il consolidamento termina quando il relativo insieme di impatti è stato propagato e verificato.

## 22.1 Che cos'è PROC-006

PROC-006 mantiene coese la **Working Memory** e la **Persistent Organizational Memory**.

Non trasforma automaticamente ogni elemento della conversazione in conoscenza persistente. Identifica invece i delta che devono sopravvivere, ne verifica authority e status, individua il target persistente corretto e controlla le dipendenze materiali coinvolte.

La sequenza canonica può essere letta così:

```
DELTA
↓
CLASSIFICAZIONE
↓
AUTHORITY / STATUS
↓
IMPACT SET
↓
PERSISTENT TARGET
↓
PROPAGAZIONE
↓
CONSISTENCY BUNDLE CHECK
├─ PASS → consolidamento completo
└─ FAIL → MEMORY_DRIFT
```

Questo processo è il ponte operativo fra il principio della Dual Memory e una memoria organizzativa realmente utilizzabile nelle sessioni future.

## 22.2 Il delta: ciò che è davvero cambiato

PROC-006 parte da una domanda più precisa di «che cosa è successo?»:

**| che cosa è cambiato rispetto alla memoria persistente rilevante?**

Il delta può riguardare, per esempio:

- una decisione;
- uno stato o un fatto corrente;
- lo stato esecutivo di un workflow;
- un requisito, un vincolo o un rischio;
- un output approvato o frozen;
- una nuova evidenza;
- una relazione materiale;
- un apprendimento potenzialmente generalizzabile;
- un elemento diventato superseded.

Il processo non richiede invece di trasformare in memoria permanente ogni ragionamento temporaneo, ogni tentativo o ogni frase scambiata durante il lavoro.

Questa distinzione protegge la memoria da due errori opposti: **dimenticare ciò che deve sopravvivere** e **conservare indiscriminatamente ciò che non serve**.

## 22.3 Classificare prima di scrivere

Una stessa informazione può avere significati molto diversi.

Una possibilità discussa non è una decisione. Un'ipotesi non è un requisito. Un risultato osservato non è automaticamente una regola del metodo. Una versione precedente non deve necessariamente essere cancellata: può dover essere preservata come **SUPERSEDED**.

PROC-006 classifica quindi il delta prima della persistenza.

Una classificazione operativa può distinguere almeno:

TEMPORANEO / RAGIONAMENTO  
IPOTESI / PROPOSTA  
DECISIONE  
STATO / FATTO  
EXECUTION STATE / WORKFLOW CHECKPOINT  
REQUISITO / VINCOLO / RISCHIO  
EVIDENZA  
APPRENDIMENTO / METODO  
SUPERSEDED

Classificare significa stabilire **che cosa il delta rappresenta**, non soltanto dove salvarlo.

## 22.4 Authority e status: la memoria non può promuovere da sola

Prima della scrittura, PROC-006 verifica authority e forza dell'informazione.

Questo impedisce che il semplice fatto di persistere qualcosa ne cambi lo status.

PROPOSTA  
≠  
DECISIONE

EVIDENZA  
≠  
BASELINE

PERSISTENZA  
≠  
PROMOZIONE

Se il delta modifica una decisione significativa, il processo richiama la disciplina di Decision Change & Impact Analysis. Se emerge un apprendimento generalizzabile, può intradarlo verso il Learning System e l'eventuale Evidence → Baseline Promotion. Se il cambiamento è un WCM CHANGE materiale, la normale consistenza della memoria non sostituisce il relativo Change Gate e il successivo Propagation Gate.

PROC-006 consolida ciò che possiede authority per essere consolidato; non crea authority.

## 22.5 Il Persistent Target

Dopo la classificazione occorre individuare il **Persistent Target primario**: la fonte in cui quel tipo di delta deve vivere.

Esempi tipici sono:

| Tipo di delta     | Destinazione tipica                            |
|-------------------|------------------------------------------------|
| decisione         | Decision record / registro decisionale         |
| stato corrente    | stato o registro corrente                      |
| execution state   | workflow checkpoint persistente                |
| requisito         | requirements/spec pertinente                   |
| output approvato  | output canonico/frozen + indice se necessario  |
| evidenza          | evidence / telemetry / result                  |
| relazione         | ledger o typed relation pertinente             |
| apprendimento WCM | Learning System / Method KB secondo governance |
| storico superato  | lineage preservato come <b>SUPERSEDED</b>      |

Questa tabella non significa che ogni implementazione debba usare gli stessi nomi di file. Il principio canonico è che il delta venga collocato nella fonte persistente appropriata e autorevole per quel tipo di informazione.

## 22.6 Perché aggiornare un solo file può non bastare

Immaginiamo che lo stato reale sia cambiato e che il source-of-truth sia stato aggiornato correttamente. Se però un indice, una vista corrente o un checkpoint dipendente continua a mostrare il vecchio stato, la memoria organizzativa diventa internamente incoerente.

Per questo PROC-006 costruisce un **Impact Set minimo**.

L'Impact Set risponde alla domanda:

**quali nodi dipendono materialmente da questo delta e devono essere verificati o aggiornati?**

Può comprendere:

```
CURRENT-FACING MIRRORS
WORKFLOW CHECKPOINTS
INDEX
DECISION / SOURCE REGISTERS
LIVING LEDGERS
RELATIONSHIPS / SYNAPSES
DOCUMENTATION PROJECTIONS
```

L'obiettivo non è aggiornare tutto il repository. È identificare soltanto le dipendenze che potrebbero diventare false o stale a causa del delta.

## 22.7 Il workflow checkpoint fa parte della memoria operativa

Quando cambia lo stato di un workflow materiale, il checkpoint persistente entra nell'Impact Set.

Possono cambiare elementi come:

```
STATUS
LAST_COMPLETED_TRANSITION
NEXT_TRANSITION
TRUE_STOP_CONDITION
RESUME_REQUIRED
COMPLETED_STEP_IDS
```

Se una transizione è stata completata ma il checkpoint continua a rappresentarla come pendente, una sessione successiva può ripetere lavoro già svolto o ricostruire uno stato falso.

Per questo il checkpoint deve essere aggiornato subito dopo la transizione materiale quando il workflow lo richiede.

La memoria organizzativa non contiene soltanto conoscenza descrittiva. Contiene anche la continuità necessaria a sapere **dove riprendere**.

---

## 22.8 Preservare lineage invece di riscrivere il passato

La consistenza non richiede di cancellare ogni versione precedente.

Quando un elemento viene superato, il sistema deve poter distinguere:

```
CURRENT
```

da:

```
SUPERSEDED
```

Preservare il lineage permette di ricostruire come si è arrivati allo stato corrente, quale decisione ha sostituito la precedente e quali evidenze hanno contribuito al cambiamento.

Una memoria che conserva soltanto l'ultima fotografia può sembrare pulita, ma perde spiegabilità. Una memoria che conserva tutto senza status diventa invece ambigua.

PROC-006 cerca il punto intermedio: **storia preservata, stato corrente distinguibile**.

---

## 22.9 Le sinapsi fanno parte della consistenza

Un delta può cambiare non soltanto un nodo, ma anche le sue relazioni.

Se una decisione sostituisce una precedente, una relazione di lineage può dover essere aggiornata. Se un nuovo output implementa una baseline, la relazione corrispondente può diventare materiale. Se una dipendenza cessa di essere valida, lasciarla attiva produce una falsa mappa della conoscenza.

PROC-006 richiama quindi **PR0T-013 – Knowledge Synapse & Health Standard** quando le typed relations sono coinvolte.

La regola non è «aggiungere più collegamenti possibile».

È:

**mantenere le relazioni che hanno significato operativo o causale e che servono a ricostruire correttamente la conoscenza.**

Le sinapsi decorative aumentano rumore senza aumentare affidabilità.

---

## 22.10 Il Consistency Bundle Check

La scrittura del delta non conclude il processo. Prima della closure viene eseguito il **Consistency Bundle Check**.

Il controllo verifica almeno domande come:

1. qual è il nodo source-of-truth del delta?
2. esiste un workflow checkpoint coinvolto?
3. stato, ultima transizione, next transition e true stop sono coerenti?
4. quali current-facing file dichiarano lo stesso stato o authority?
5. quali indici, registri o ledger dipendono dal delta?
6. quali relazioni devono cambiare?
7. esistono due rappresentazioni correnti incompatibili?
8. uno step completato appare ancora pendente?
9. il Knowledge Health Check copre l'ultimo delta materiale?

Il bundle non è un controllo cosmetico. Serve a dimostrare che la memoria persistente può essere usata da una sessione futura senza richiedere la chat precedente per capire quale versione sia vera.

---

## 22.11 PASS e MEMORY\_DRIFT

Il Consistency Bundle Check produce una distinzione netta.

BUNDLE VERDE  
→ CONSOLIDAMENTO COMPLETO

oppure:

BUNDLE NON VERDE  
→ MEMORY\_DRIFT  
→ PROC-008 / KNOWLEDGE STEWARD SE APPLICABILE

Un **MEMORY\_DRIFT** non significa necessariamente che il source-of-truth sia sbagliato. Può significare che una sua proiezione, un indice, un ledger o una relazione non sono più coerenti.

La regola importante è che il sistema **non dichiari completo il consolidamento** mentre conosce una divergenza materiale non risolta.

---

## 22.12 Knowledge Health e freshness

La consistenza non riguarda soltanto il contenuto, ma anche la freschezza delle verifiche.

Se l'ultimo Knowledge Health Check precede l'ultimo delta materiale, quel controllo non può essere usato come prova che la memoria successiva al delta sia ancora verde.

In forma semplice:

```
KNOWLEDGE CHECK  
precedente al  
MATERIAL DELTA
```

→ non prova HEALTHY dopo il delta

PROC-006 richiede quindi che l'assurance sia corrente oppure che lo stato della knowledge health dichiari esplicitamente la condizione non-green pertinente fino a nuova verifica.

Non si tratta di sostenere che ogni repository debba avere un unico meccanismo universale di health checking. È una regola della baseline WCM corrente per evitare che una verifica vecchia certifichi implicitamente uno stato nuovo.

---

## 22.13 Relazione con il Completion Gate

PROC-006 è una preconditione della closure di un workflow materiale quando quel workflow ha prodotto delta persistenti.

Il caso vietato è concettualmente questo:

```
SOURCE OF TRUTH AGGIORNATA  
+  
MIRROR STALE  
+  
CHECKPOINT NON CORRENTE  
↓  
PROC-006 FAIL  
↓  
COMPLETED VIETATO
```

La fine della sessione non sostituisce questo controllo.

La closure diventa possibile quando ciò che deve sopravvivere è persistito, l'Impact Set è stato propagato, i current-facing mirrors pertinenti non sono in conflitto e una sessione futura può ricostruire stato e next transition senza dipendere dalla conversazione precedente.

---

## 22.14 Il confine del Knowledge Steward

Il Knowledge Steward può eseguire riparazioni meccaniche o strutturali quando il significato corretto è già determinato dalle fonti autorevoli.

Per esempio, può essere possibile riallineare un riferimento rotto o una proiezione deterministica se non esiste ambiguità semantica.

Ma se il conflitto richiede di decidere **che cosa dovrebbe significare** una fonte, il Knowledge Steward non può inventare la soluzione.

DRIFT STRUTTURALE DETERMINABILE  
→ repair consentito secondo governance

CONFLITTO SEMANTICO  
→ NO WRITE  
→ escalation

Lo stesso vale per `next_transition`, true stop condition o authority: se non sono determinabili dalle fonti autorevoli, non possono essere creati per rendere artificialmente verde il bundle.

## 22.15 Consolidation non è promotion

PROC-006 e PROC-004 risolvono problemi diversi.

PROC-006 chiede:

**|** *che cosa deve essere persistito e propagato affinché la memoria rimanga coerente?*

PROC-004 chiede:

**|** *una determinata evidenza merita di modificare una baseline governata?*

Un'evidenza può essere consolidata come evidenza senza diventare regola. Un learning può essere registrato senza essere promosso. Una proposta può essere ricordata come proposta senza diventare decisione.

Questa separazione evita che il semplice atto di «ricordare» diventi un canale implicito di modifica del metodo.

## 22.16 Consolidation e WCM CHANGE

Quando il delta è un WCM CHANGE materiale, PROC-006 resta necessario ma non è sufficiente per dichiarare il cambiamento chiuso.

Il bundle verde dimostra che la memoria coinvolta dal delta è coerente. Non dimostra da solo che l'intero cambiamento di metodo sia stato autorizzato, propagato e chiuso secondo governance.

Per un WCM CHANGE materiale la catena prosegue verso **PROC-012** / **PROT-019** e il relativo Propagation Gate.

PROC-006 PASS  
≠



Questa distinzione impedisce che un aggiornamento tecnicamente coerente venga scambiato per una modifica governata e approvata del metodo.

## 22.17 Failure mode

PROC-006 considera failure mode comportamenti come:

- copiare indiscriminatamente una conversazione nella knowledge base;
- non consolidare una decisione perché «è già nel contesto»;
- trasformare una proposta in decisione durante la persistenza;
- sovrascrivere una decisione senza lineage;
- aggiornare soltanto il source-of-truth ignorando dipendenze materiali note;
- lasciare un entry point o un mirror corrente in stato stale;
- dimenticare il workflow checkpoint dopo una transizione materiale;
- dichiarare **COMPLETED** con consistency bundle non verde;
- aggiungere sinapsi decorative;
- duplicare la stessa informazione in fonti non governate;
- rappresentare come **HEALTHY** uno stato non coperto da un check successivo all'ultimo delta materiale.

La radice comune è confondere **scrittura** con **consistenza**.

## 22.18 Output del processo

PROC-006 non impone la creazione di un file separato per ogni consolidamento. L'Impact Set deve però essere ricostruibile quando il delta è materiale.

Una rappresentazione possibile è:

```
MEMORY_DELTA:
  added: []
  changed: []
  superseded: []
  unresolved: []
  persistent_target:
  workflow_instance_id:
  impact_set:
    current_state_mirrors: []
    workflow_checkpoints: []
    indexes: []
    decision_or_source_registers: []
    living_ledgers: []
    relationships: []
    documentation_projections: []
  assurance_required: true|false
```

Il formato è un modello operativo raccomandato, non la creazione di una nuova regola editoriale o di un nuovo schema universale.

Il vero output è una memoria persistente in cui il delta necessario è stato registrato e le sue dipendenze materiali sono state verificate.

---

## 22.19 Relazioni con gli altri processi e protocolli

PROC-006 è strettamente collegato a:

- **PROC-005 – Agent-Ready Context Bootstrap**, perché una memoria consolidata permette alla sessione successiva di ricostruire il contesto;
- **PROT-009 – Contiguous Workflow Execution**, perché il Completion Gate dipende dalla corretta persistenza dei delta materiali;
- **PROT-007 – Decision Change & Impact Analysis**, quando cambia una decisione significativa;
- **PROT-013 – Knowledge Synapse & Health Standard**, quando cambiano relazioni o health;
- **PROC-008 – Knowledge Integrity Assurance Loop**, quando il bundle rileva drift;
- **PROC-004 – Evidence → Baseline Promotion**, quando un'evidenza o learning può avere impatto sulla baseline;
- **PROC-012 / PROT-019**, quando il delta appartiene a un WCM CHANGE materiale.

Il processo non sostituisce questi elementi. Coordina il passaggio dalla memoria viva alla memoria persistente e consegna agli altri processi i casi che richiedono una governance specifica.

---

## 22.20 Maturità e limiti

Il processo canonico è classificato **VALIDATED BY GOVERNANCE / SESSION-INDEPENDENT EVOLUTION ACTIVE / FIELD VALIDATION IN PROGRESS**.

La baseline corrente definisce delta detection, classificazione, authority/status check, Impact Set, workflow checkpoint, consistency bundle e gestione del memory drift.

Questa maturità non equivale a sostenere che ogni possibile dominio, repository, tipo di relazione o configurazione organizzativa sia già stato validato sul campo.

PROC-006 non promette inoltre che ogni conflitto possa essere risolto automaticamente. Dove il significato non è determinabile, il sistema deve preservare il confine tra repair strutturale e decisione semantica.

---

## 22.21 La regola da ricordare

Se dovessimo condensare PROC-006 in una sola idea, sarebbe questa:

**la memoria è consolidata non quando qualcosa è stato scritto, ma quando ciò che deve sopravvivere è persistito con il giusto status e le dipendenze materiali sono tornate coerenti.**

È il passaggio che consente alla Dual Memory di funzionare nel tempo senza trasformare la memoria persistente né in un deposito indiscriminato né in una collezione di fotografie contraddittorie.

---

## Source Map

Fonti canoniche principali utilizzate per il Technical Truth Pass:

- `wcm/process-book/PROCESS_REGISTER.md`;
- `wcm/process-book/processes/PROC-006_MEMORY_CONSOLIDATION_LOOP.md`;
- `WCM_AGENT_START.md` per la relazione con bootstrap, Completion Gate e source precedence.

Nessuna nuova regola WCM è introdotta da questo capitolo. Gli esempi e le formulazioni sono pedagogici e domain-agnostic.